Lab 11

Introduction

In operating systems, memory allocation is a critical task. When processes request memory dynamically, the OS must decide where in memory to place each process. Poor allocation strategies can lead to external fragmentation, small, unusable gaps in memory.

Three common memory allocation strategies are:

1. **First Fit:** Allocate the first block that is large enough. (Provided)
2. **Best Fit:** Allocate the smallest block that is large enough.
3. **Worst Fit:** Allocate the largest block available.

The goal of this lab exercise is to:

- Implement Best Fit and Worst Fit algorithms.
- Compare the resulting memory fragmentation for a given workload.
- Analyze trade-offs between these strategies.

Algorithm Explanations

1. Best Fit

Logic:

- For each process, search the entire list of free memory blocks.
- Select the block that is large enough for the process and leaves the smallest leftover fragment.
- If multiple blocks are equally good, choose the first one found.

Advantages:

- Minimizes wasted space in individual blocks.
- Can be efficient for small processes.

Disadvantages:

- May create many tiny fragments that cannot be used for future processes.
- Searching all blocks can be slower.

2. Worst Fit

Logic:

- For each process, search the entire list of free memory blocks.
- Select the block that is large enough and leaves the largest leftover fragment.
- If multiple blocks are equally large, choose the first one found.

Advantages:

- Keeps large blocks free for future processes.
- Can reduce the chance of small, unusable fragments.

Disadvantages:

- May waste memory unnecessarily by leaving very large fragments unused for small processes.

Best Fit:

```
hp@TF-COMP-786:~/my_project$ cat > memory_allocation.py
import copy
class MemoryBlock:
    def __init__(self, size, is_free=True, process_id=None):
        self.size = size
        self.is_free = is_free
        self.process_id = process_id

class Process:
    def __init__(self, pid, size):
        self.pid = pid
        self.size = size

def print_memory_layout(memory):
    print("\n--- Memory Layout ---")
    layout = "|"
    for block in memory:
        if block.is_free:
            layout += f" Free({block.size}K) |"
        else:
            layout += f" P{block.process_id}({block.size}K) |"
    print(layout)
    print("-" * len(layout))

def best_fit(initial_holes, processes):
    print("\n\n>>>--- Starting Best Fit Simulation ---<<<")
    memory = [MemoryBlock(size=h) for h in initial_holes]
    for process in processes:
        allocated = False
        best_idx = None
        best_size_diff = float('inf')
        for i, block in enumerate(memory):
            if block.is_free and block.size >= process.size:
                size_diff = block.size - process.size
                if size_diff < best_size_diff:
                    best_size_diff = size_diff
                    best_idx = i
        if best_idx is not None:
            block = memory[best_idx]
            print(f"Allocating Process P{process.pid} (size {process.size}K)"
                  "to best-fit hole of size {block.size}K.")
            if block.size > process.size:
                remaining_size = block.size - process.size
                new_block = MemoryBlock(size=remaining_size)
                memory.insert(best_idx + 1, new_block)
            block.size = process.size
            block.is_free = False
            block.process_id = process.pid
            allocated = True
        if not allocated:
            print(f"Process P{process.pid} (size {process.size}K) could not be allocated.")

    print_memory_layout(memory)
    return memory
```

Worst Fit

```
def worst_fit(initial_holes, processes):
    print("\n\n>>>--- Starting Worst Fit Simulation ---<<<")
    memory = [MemoryBlock(size=h) for h in initial_holes]
    for process in processes:
        allocated = False
        worst_idx = None
        worst_size_diff = -1
        for i, block in enumerate(memory):
            if block.is_free and block.size >= process.size:
                size_diff = block.size - process.size
                if size_diff > worst_size_diff:
                    worst_size_diff = size_diff
                    worst_idx = i
        if worst_idx is not None:
            block = memory[worst_idx]
            print(f"Allocating Process P{process.pid} (size {process.size}K) "
                  " to worst-fit hole of size {block.size}K.")
            if block.size > process.size:
                remaining_size = block.size - process.size
                new_block = MemoryBlock(size=remaining_size)
                memory.insert(worst_idx + 1, new_block)
            block.size = process.size
            block.is_free = False
            block.process_id = process.pid
            allocated = True

        if not allocated:
            print(f"Process P{process.pid} (size {process.size}K) could not be allocated.")

    print_memory_layout(memory)
    return memory
```

First Fit

```
def first_fit(initial_holes, processes):
    print("\n\n>>>--- Starting First Fit Simulation ---<<<")
    memory = [MemoryBlock(size=h) for h in initial_holes]

    for process in processes:
        allocated = False
        for i, block in enumerate(memory):
            if block.is_free and block.size >= process.size:
                print(f"Allocating Process P{process.pid} (size {process.size}K) to first available hole of size {block.size}K.")

                # Split the block if it is bigger than needed
                if block.size > process.size:
                    remaining_size = block.size - process.size
                    new_block = MemoryBlock(size=remaining_size)
                    memory.insert(i + 1, new_block)

                block.size = process.size
                block.is_free = False
                block.process_id = process.pid
                allocated = True
                break  # Move to next process

        if not allocated:
            print(f"Process P{process.pid} (size {process.size}K) could not be allocated.")

    print_memory_layout(memory)
    return memory
```

```python
def analyze_algorithms_table(holes, processes):
    results = []

    # First Fit
    ff_memory = first_fit(holes, processes)
    total_free = sum(b.size for b in ff_memory if b.is_free)
    largest_free = max((b.size for b in ff_memory if b.is_free), default=0)
    results.append(("First Fit", total_free, largest_free))

    # Best Fit
    bf_memory = best_fit(holes, processes)
    total_free = sum(b.size for b in bf_memory if b.is_free)
    largest_free = max((b.size for b in bf_memory if b.is_free), default=0)
    results.append(("Best Fit", total_free, largest_free))

    # Worst Fit
    wf_memory = worst_fit(holes, processes)
    total_free = sum(b.size for b in wf_memory if b.is_free)
    largest_free = max((b.size for b in wf_memory if b.is_free), default=0)
    results.append(("Worst Fit", total_free, largest_free))

    # Print table
    print("\n========= Memory Allocation Comparison =========")
    print(f"{'Algorithm':<12} | {'Total Free (K)':<15} | {'Largest Free Block (K)':<20}")
    print("-" * 55)
    for alg, total, largest in results:
        print(f"{alg:<12} | {total:<15} | {largest:<20}")
    print("-" * 55)
```

```python
if __name__ == "__main__":
# Define initial memory state and processes
    initial_memory_holes = [100, 500, 200, 300, 600] # Sizes in KB
    processes_to_allocate = [
    Process(1, 212), Process(2, 417), Process(3, 112), Process(4, 426)
    ]
    analyze_algorithms_table(initial_memory_holes,processes_to_allocate)
    # # Run the First Fit simulation
    # first_fit_memory = first_fit(copy.deepcopy(initial_memory_holes), processes_to_allocate)
    # best_fit_memory = best_fit(copy.deepcopy(initial_memory_holes), processes_to_allocate)
    # worst_fit_memory = worst_fit(copy.deepcopy(initial_memory_holes), processes_to_allocate)
bp@IF-COMP-786:~/my_project$ python3 memory_allocation.py
```

```
>>>--- Starting First Fit Simulation ---<<<
Allocating Process P1 (size 212K) to first available hole of size 500K.
Allocating Process P2 (size 417K) to first available hole of size 600K.
Allocating Process P3 (size 112K) to first available hole of size 288K.
Process P4 (size 426K) could not be allocated.

--- Memory Layout ---
| Free(100K) | P1(212K) | P3(112K) | Free(176K) | Free(200K) | Free(300K) | P2(417K) | Free(183K) |
-------------------------------------------------------------------------------------------------


>>>--- Starting Best Fit Simulation ---<<<
Allocating Process P1 (size 212K)to best-fit hole of size {block.size}K.
Allocating Process P2 (size 417K)to best-fit hole of size {block.size}K.
Allocating Process P3 (size 112K)to best-fit hole of size {block.size}K.
Allocating Process P4 (size 426K)to best-fit hole of size {block.size}K.

--- Memory Layout ---
| Free(100K) | P2(417K) | Free(83K) | P3(112K) | Free(88K) | P1(212K) | Free(88K) | P4(426K) | Free(174K) |
-------------------------------------------------------------------------------------------------------------


>>>--- Starting Worst Fit Simulation ---<<<
Allocating Process P1 (size 212K)  to worst-fit hole of size {block.size}K.
Allocating Process P2 (size 417K)  to worst-fit hole of size {block.size}K.
Allocating Process P3 (size 112K)  to worst-fit hole of size {block.size}K.
Process P4 (size 426K) could not be allocated.

--- Memory Layout ---
| Free(100K) | P2(417K) | Free(83K) | Free(200K) | Free(300K) | P1(212K) | P3(112K) | Free(276K) |
-------------------------------------------------------------------------------------------------


========= Memory Allocation Comparison =========
Algorithm    | Total Free (K)  | Largest Free Block (K)
-------------------------------------------------------
First Fit    | 959             | 300
Best Fit     | 533             | 174
Worst Fit    | 959             | 300
-------------------------------------------------------
```

Key Observations:

- **Best Fit** reduces leftover space in individual blocks but can create many small unusable fragments (external fragmentation).
- **Worst Fit** preserves large contiguous blocks, which can be useful for future large processes, but may waste memory for small processes.
- **Trade-off:** Best Fit → efficient for minimizing immediate waste, Worst Fit → efficient for **l**ong-term large block availability.

Conclusion

- **Most successful at preserving large contiguous blocks:** Worst Fit, because it leaves larger holes for future allocations.
- **Most external fragmentation: Best Fit**, due to many small leftover fragments.
- **Primary trade-off:**
    - Best Fit: minimizes waste in each allocation but can fragment memory overall.
    - Worst Fit: reduces fragmentation for large processes but may waste memory on small allocations.